

[3조] 모터 제어 상위설계서

신유지, 이호윤, 이양배, 한소영

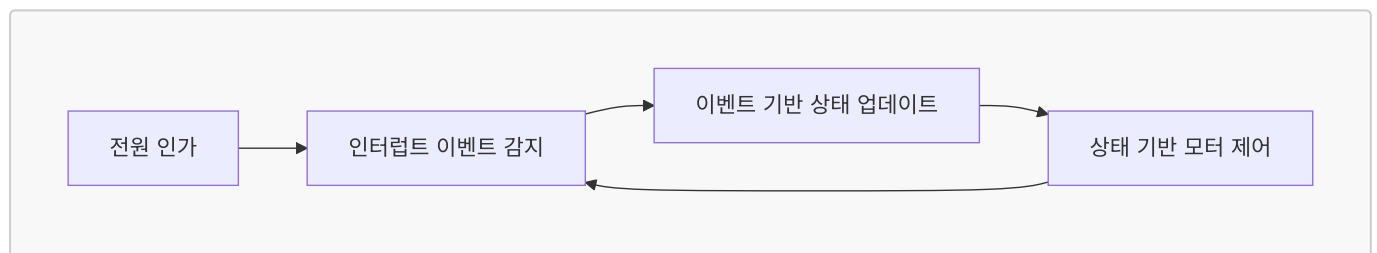
요구 사항

- 모터는 버튼과 UART 명령 두 가지 방식으로 제어할 수 있어야 한다.
- 시스템 시작 시 모터는 정지 상태여야 한다.
- 메인 루프는 UART 처리, 버튼 처리, 모터 상태 처리를 반복 수행한다.
- 방향 전환시 모터 부하 방지를 위해 전환 간 대기시간을 둔다.
- UART PWM duty rate 50 ~ 95% 이다.
- 위 요구사항에 명시되지 않은 세부 구현 방식은 자유롭게 설계한다.

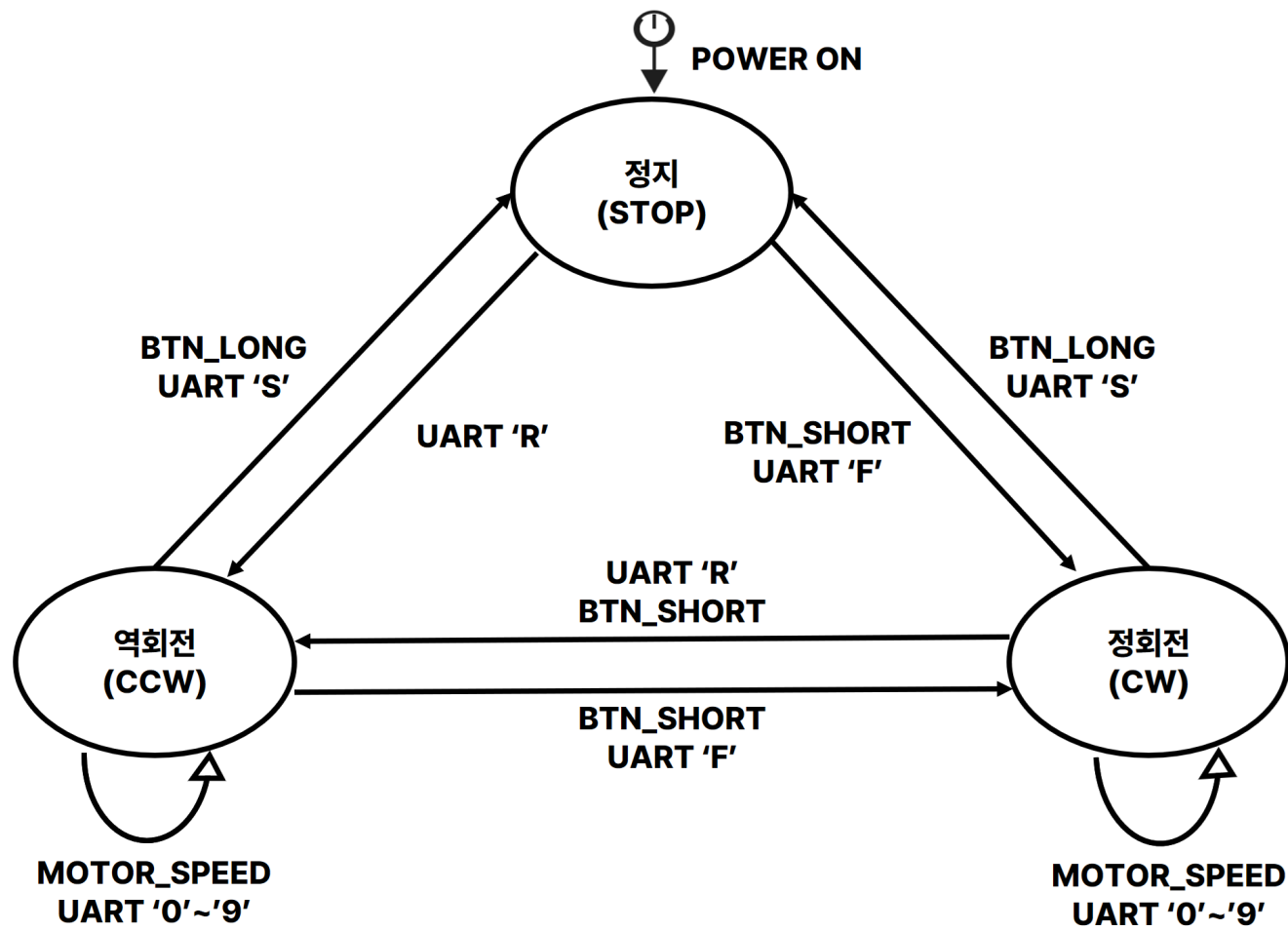
1. STATE

1-1. FSM

다이어그램 1. 메인 동작 흐름



다이어그램 2. 모터 상태 전이



1-2. MOTOR STATE

이름	값	설명
STOP	0	모터 정지
CW	1	모터 정회전
CCW	2	모터 역회전

1-3. MOTOR SPEED

이름	값	설명
0	50	PWM Duty 50% 설정
1	55	PWM Duty 55% 설정
2	60	PWM Duty 60% 설정
3	65	PWM Duty 65% 설정
4	70	PWM Duty 70% 설정
5	75	PWM Duty 75% 설정
6	80	PWM Duty 80% 설정

이름	값	설명
7	85	PWM Duty 85% 설정
8	90	PWM Duty 90% 설정
9	95	PWM Duty 95% 설정

2. EVENT (BUTTON/UART)

2-1. BUTTON/UART 이벤트

이름	값	설명
EVT_NONE	0	이벤트 없음 / 이벤트 소비 완료
EVT_BTN_SHORT	1	버튼 짧게 눌림 (버튼 눌린 시점부터 떴을 시간까지 측정. 눌린 시간 < 3000ms)
EVT_BTN_LONG	2	버튼 길게 눌림 (버튼이 눌린 채로 경과 시간 >= 3000ms)
EVT_UART	3	UART로 문자 수신 (수신 문자는 ISR에서 Uart_Data_In에 저장)

2-2. 상태별 이벤트 처리표

현재 상태	입력 이벤트/명령	다음 상태	수행 함수	전환 전 정지 지연
MOTOR_STOP	버튼 짧게 누름	MOTOR_CW	motor_cw()	없음
MOTOR_STOP	버튼 길게 누름	MOTOR_STOP	없음	없음
MOTOR_CW	버튼 짧게 누름	MOTOR_CCW	motor_stop_delay() → motor_ccw()	적용
MOTOR_CW	버튼 길게 누름	MOTOR_STOP	motor_stop()	없음
MOTOR_CCW	버튼 짧게 누름	MOTOR_CW	motor_stop_delay() → motor_cw()	적용
MOTOR_CCW	버튼 길게 누름	MOTOR_STOP	motor_stop()	없음
모든 상태	UART 'f'	MOTOR_CW	motor_cw()	미적용
모든 상태	UART 's'	MOTOR_STOP	motor_stop()	없음
모든 상태	UART 'r'	MOTOR_CCW	motor_ccw()	미적용
모든 상태	UART '0'~'9'	현재 상태 유지	TIM5_Out_PWM_Generation()	해당 없음

3. PWM 및 타이머

3-1. TIM5 모터 PWM

항목	설정값
TIM5 기준 주파수	8 MHz
PWM 주파수	10 kHz
출력 채널	CH1, CH2
카운터 모드	Down counter
반복 모드	Repeat
초기 Duty	75% (range = 5)
Duty 범위	50~95%

3-2. TIM4 버튼 3초 인지 타이머

항목	설정값
TIM4 Tick	1000 μ s
TIM4 기준 주파수	1 kHz
길게 누름 설정값	3000 ms
인터럽트	Update interrupt, IRQ 30

4. VARIABLE

4-1. 열거형 및 구조체

```
typedef enum
{
    MOTOR_STOP,
    MOTOR_CW,
    MOTOR_CCW
} MOTOR_STATE;

typedef enum
{
    BTN_NONE,
    BTN_SHORT_PRESSED,
    BTN_LONG_PRESSED,
    BTN_UART
} BUTTON_EVENT;

typedef struct Button
{
    BUTTON_EVENT event;
    int pressed;
    int long_pressed;
} BUTTON;
```

4-2. GLOBAL VARIABLE

이름	Type	초기값	역할	변경 시점
evt	event_t	EVT_NONE	현재 이벤트 저장	BUTTON, UART 인터럽트 발생시 <code>check_event()</code> 에서 갱신
motor_state	motor_state_t	MOTOR_STOP	현재 모터 상태 저장	<code>drive_motor()</code> 에서 모터 상태 변경 시 갱신
new_motor_state	motor_state_t	MOTOR_STOP	새로운 모터 회전 저장	<code>update_motor_state()</code> 에서 이벤트 처리 시 갱신
motor_speed	uint8_t	5	현재 PWM Duty 레벨 (0~9)	<code>drive_motor()</code> 에서 모터 상태 변경 시 갱신
new_motor_speed	uint8_t	5	새로운 PWM Duty 레벨 (0~9)	<code>update_motor_state()</code> 에서 이벤트 처리 시 갱신
btn_flag	volatile uint8_t	0	버튼 인터럽트 (falling edge/rising edge) 발생 여부	Button_ISR에서 1로 세팅 → <code>check_event()</code> 에서 읽은 직후 0으로 리셋
uart_flag	volatile uint8_t	0	UART 인터럽트 발생 여부	<code>USART2_IRQHandler()</code> 에서 1로 세팅 → <code>check_event()</code> 에서 읽은 직후 0으로 리셋
uart_data	volatile uint8_t	0	UART로 수신된 문자	<code>USART2_IRQHandler()</code> 에서 수신 시 갱신
timer_flag	volatile uint8_t	0	타이머 인터럽트 발생 여부	<code>TIM4_IRQHandler()</code> 에서 갱신

5. METHOD

5-1. 주요 METHOD

이름	입력	출력	설명	호출 시점
<code>check_event()</code>	btn_flag, uart_flag (전역 참조)	evt 갱신	btn_flag가 세워져 있으면 핀 상태로 falling/rising을 판별해 EVT_BTN_SHORT 여부를 계산하고, 눌린 채로 3초가 지나면 EVT_BTN_LONG을 생성. uart_flag가 세워져 있으면 EVT_UART를 생성.	main 루프

이름	입력	출력	설명	호출 시점
update_motor_state()	evt, motor_state, motor_speed (전역 참조)	new_motor_state, new_motor_speed 갱신	evt가 EVT_BTN_SHORT면 CW/CCW를 토글, EVT_BTN_LONG이면 MOTOR_STOP으로 전환. EVT_UART면 uart_data가 숫자인지 'f', 'r', 's'인지에 따라 new_motor_speed 또는 new_motor_state를 갱신.	main 루프, evt != EVT_NONE 일 때만 호출
drive_motor()	new_motor_state, new_motor_speed (전역 참조)	GPIO 방향 설정, PWM Duty 반영, motor_state, motor_speed 갱신	new_motor_state와 motor_state, new_motor_speed와 motor_speed를 비교하여 다를 때만 GPIO 방향을 설정하고 PWM Duty를 반영	main 루프

5-2. ISR

이름	출력	설명	호출 시점
EXTI15_10_IRQHandler()	btn_flag 갱신	btn_flag를 1로 세팅	PC13 USER KEY (falling edge, rising edge) 인터럽트 발생
USART2_IRQHandler()	uart_flag 갱신	입력 받은 문자를 uart_data에 저장하고, uart_flag를 1로 세팅	USART2 인터럽트 발생
TIM4_IRQHandler()	timer_flag 갱신	timer_flag를 1로 세팅	타이머 인터럽트(3000ms) 발생